

SITUAÇÃO DE APRENDIZAGEM	
Segmento:	Técnico
Unidade (s) Curricular (es):	Lógica de Programação
Curso (s):	Desenvolvimento de Sistemas
Instrutor (a):	Vitor Hugo Almeida de Sousa

ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA	
☒ Situação-problema ☐ Estudo do Caso ☐ Projeto ☐ Pesquisa Aplicada	

DESCRIÇÃO DA ESTRATÉGIA DE APRENDIZAGEM DESAFIADORA
Contextualização: Herança em POO 2 Uma equipe de desenvolvimento está criando um sistema simples para controlar máquinas utilizadas em uma indústria. O sistema deve ser estruturado de forma orientada a objetos, permitindo organizar melhor o código e reaproveitar informações comuns entre diferentes tipos de máquinas. Durante o desenvolvimento, percebeu-se que diferentes máquinas industriais possuem algumas características em comum, como nome, setor e status de funcionamento. Porém, cada tipo de máquina também possui informações específicas. Para isso, será utilizado o conceito de herança. Desenvolva um programa chamado ProjetoIndustria, utilizando o pacote industria, tendo como arquivo principal a classe App.java, com o método main. Todas as classes deverão ser criadas em arquivos separados dentro do mesmo pacote. Nesse programa, você deverá criar uma classe base chamada Maquina e duas classes derivadas chamadas MaquinaCorte e MaquinaSolda. A classe Maquina deverá possuir os atributos private: nome, setor e status. Implemente os métodos getters e setters para todos os atributos, com as seguintes validações: O nome não pode ser vazio. O setor não pode ser vazio. O status não pode ser vazio. Além disso, a classe Maquina deverá possuir um método chamado exibirDadosBasicos, responsável por mostrar o nome, o setor e o status da máquina. A classe MaquinaCorte deverá herdar de Maquina usando extends Maquina e possuir o atributo private: tipoCorte. Esse atributo deverá representar o tipo de processo de corte realizado pela máquina. Alguns exemplos de valores que podem ser preenchidos são: Laser, Plasma, Guilhotina, Serra e Jato de água. Implemente o getter e o setter desse atributo, garantindo que o tipo de corte não seja vazio.

Essa classe deverá possuir um método chamado **exibirMaquinaCorte**, que mostre todos os dados da máquina, incluindo os dados herdados e o tipo de corte.

A classe **MaquinaSolda** deverá herdar de **Maquina** usando `extends Maquina` e possuir o atributo `private: tipoSolda`.

Esse atributo deverá representar o tipo de processo de soldagem realizado pela máquina. Alguns exemplos de valores que podem ser preenchidos são: **MIG**, **TIG**, **Eletrodo revestido**, **Ponto** e **Arco submerso**.

Implemente o getter e o setter desse atributo, garantindo que o tipo de solda não seja vazio.

Essa classe deverá possuir um método chamado **exibirMaquinaSolda**, que mostre todos os dados da máquina, incluindo os dados herdados e o tipo de solda.

Além disso, você deverá criar uma classe chamada **OrdemServico**, também em arquivo separado, contendo os atributos `private: responsavel` e `maquina`.

O atributo `maquina` deverá ser do tipo **Maquina**, permitindo que a ordem de serviço receba tanto um objeto da classe **MaquinaCorte** quanto um objeto da classe **MaquinaSolda**.

Implemente os getters e setters com as seguintes validações:

O responsável não pode ser vazio.

A máquina não pode ser nula.

A classe **OrdemServico** deverá possuir um método chamado **exibirResumoOrdem**, que mostre o nome do responsável e os dados básicos da máquina utilizada.

No arquivo **App.java**, você deverá instanciar pelo menos um objeto da classe **MaquinaCorte** e um objeto da classe **MaquinaSolda**, atribuindo valores por meio dos setters.

Exemplo de preenchimento para uma máquina de corte:

```
maquinaCorte.setNome("Cortadora CNC 01");
maquinaCorte.setSetor("Corte");
maquinaCorte.setStatus("Em operação");
maquinaCorte.setTipoCorte("Laser");
```

Exemplo de preenchimento para uma máquina de solda:

```
maquinaSolda.setNome("Soldadora MIG 02");
maquinaSolda.setSetor("Soldagem");
maquinaSolda.setStatus("Disponível");
maquinaSolda.setTipoSolda("MIG");
```

Em seguida, deverá criar pelo menos dois objetos da classe **OrdemServico**, um para cada tipo de máquina, e chamar o método **exibirResumoOrdem** para mostrar os dados no console.

O objetivo deste exercício é praticar herança, encapsulamento, validações simples, reutilização de código e associação entre objetos, percebendo que um atributo do tipo da classe pai pode receber objetos de classes filhas.

--

DESAFIO
Desenvolver, como programador, o sistema ProjetoIndustria , implementando corretamente a estrutura de classes, validações, herança e comportamento dos objetos para garantir o funcionamento adequado do cadastro de produtos e processamento de pedidos.

RESULTADO/ENTREGAS E ESPERADOS
Código-fonte completo e funcional do sistema ProjetoLoja, organizado em classes separadas no pacote comercio.

<i>Material de Consumo:</i>	
<i>Ferramentas:</i>	Eclipse IDE
<i>EPIs:</i>	Não se aplica
<i>EPCs:</i>	Não se aplica
<i>Equipamentos:</i>	Computador
<i>Instrumentos:</i>	Não se aplica